

SWARM-LLM: Collaborative Inference for Edge-based Small Language Models

Mostafa Dahshan, Quazi Mamun, and Tanmoy Debnath

School of Computing, Mathematics and Engineering, Charles Sturt University, NSW, Australia.

Email: {mdahshan, qmamun, tdebnath}@csu.edu.au

Abstract—Large language models (LLMs) provide strong performance across a wide range of tasks but are typically hosted on centralised cloud infrastructure, incurring significant bandwidth, latency, and privacy costs. In contrast, small language models (SLMs) can run on edge devices but have limited capability and robustness. This paper introduces SWARM-LLM, a routing and collaboration layer that coordinates a small swarm of edge-hosted SLMs with an optional cloud foundation model (FM). SWARM-LLM decides, for each query, whether to answer locally, collaborate with peer SLMs, or “summon” a cloud FM, using lightweight uncertainty estimates and safety signals. We implement a working prototype on commodity hardware with three heterogeneous SLMs and a 70B-parameter cloud FM accessed via API, and evaluate it on a controlled study workload of easy, hard, and safety-oriented queries. Our results show that SWARM-LLM substantially improves performance on hard questions compared to an edge-only deployment, while limiting cloud usage to roughly one quarter of queries, illustrating a practical trade-off between accuracy, latency, and cost for privacy-conscious edge deployments. The implementation code is available at the GitHub repository https://github.com/mdahshan/swarm_llm.

Index Terms—Edge AI, language models, collaborative swarm inference, large language model routing, uncertainty-aware decision making, privacy, cost-aware inference

I. INTRODUCTION

Large Language Models (LLMs) and other foundation models (FMs) have become a common interface for search, coding, and decision support, yet the dominant deployment pattern is still cloud-centric: user prompts are sent to a large, centralised FM (e.g., GPT-class and 70B-scale models) for every request [1], [2]. This paradigm is increasingly mismatched with the emerging landscape of pervasive, resource-constrained edge devices [3]. In practice, wide-area network (WAN) variability inflates end-to-end latency, cloud inference costs scale with token usage [4], and sending raw prompts and context off-premises can violate privacy, governance, or data-sovereignty requirements.

Meanwhile, modern edge hardware can run quantised Small Language Models (SLMs), but SLMs do not reliably handle the long tail of intrinsically hard queries or safety-sensitive

requests [5]. This raises a systems question: *how can we obtain “FM when needed” quality while keeping most queries local for latency, cost, and privacy?* Motivated by work on model routing and execution control [4], [6]–[8], we propose **SWARM-LLM**, a collaborative inference and routing layer that sits in front of a heterogeneous *swarm* of edge SLMs and an optional cloud FM endpoint. For each incoming query, SWARM-LLM estimates (i) query difficulty via lightweight uncertainty signals and (ii) safety/policy risk, then decides whether to: answer locally, consult a small set of peers and aggregate answers via consensus, or escalate to the cloud FM only when the expected utility gain justifies the added WAN delay and cloud spend.

Contributions. We make three contributions:

- We present the design of SWARM-LLM as a deployable routing and collaboration layer for heterogeneous edge SLM swarms with optional FM escalation, grounded in an explicit system model.
- We introduce lightweight difficulty (uncertainty) and safety/policy estimation together with a low-overhead, uncertainty-weighted consensus rule for combining peer answers.
- We implement a prototype on commodity hardware and evaluate the trade-offs among edge-only, cloud-only, and SWARM-LLM execution on workloads spanning easy, hard, and safety-oriented queries.

II. BACKGROUND, RELATED WORK AND DESIGN GOALS

Edge deployment of LLMs has been surveyed extensively, highlighting the core trade-offs among accuracy, latency, cost, and device constraints, and motivating hybrid edge–cloud execution rather than a purely on-device or purely cloud approach [3]. Empirical evidence also shows that smaller/quantised models can suffer factuality hallucinations on complex or long-tail prompts, reinforcing the need for selective escalation mechanisms [9]. In parallel, privacy and governance concerns require identifying and restricting sensitive content before offloading prompts or context to remote services [10].

A major line of work addresses *query routing* and *cascading*: systems such as cost/quality-aware routers and budgeted cascades decide when to invoke stronger (and more expensive) models to meet target utility under latency and spending constraints [4], [6], [8]. Separately, collaborative edge inference aims to pool capability across nearby devices via sharding or cooperative serving, improving quality without always invoking

a cloud FM [2], [11]. Finally, distillation and parameter-efficient fine-tuning (e.g., QLoRA) provide practical pathways for improving edge models over time using teacher signals [12], [13].

SWARM-LLM sits at the intersection of these threads by integrating lightweight uncertainty estimation, explicit safety/policy gating, budget-latency-aware routing, and low-overhead swarm consensus into a single deployable pipeline, with optional teacher-driven adaptation when cloud access is available.

Edge deployment of language models is increasingly attractive for interactive IoT/mobile services, where cloud-only FMs add WAN latency, recurring inference cost, and governance risk; surveys emphasise that real deployments must cope with constrained resources, heterogeneous devices, and intermittent connectivity [3], [5], [14]. Quantised SLMs can answer many routine queries locally, but are less reliable on the long tail and may hallucinate on complex prompts [9]. A common response is selective escalation via routing/cascading decision layers that optimise accuracy-latency-cost trade-offs under budgets [4], [6]–[8]. Separately, collaborative edge inference pools capability across nearby nodes (e.g., sharding or distributed experts) and collaborative serving frameworks demonstrate multi-node coordination for LLM workloads [2], [11], [15]. SWARM-LLM integrates these ideas with (i) cheap difficulty/uncertainty signals, (ii) safety/policy gating to reduce sensitive prompt exposure [10], and (iii) three-way selection among local, swarm consensus, and cloud escalation, while supporting heterogeneous SLMs and lightweight adapters via efficient fine-tuning [13].

Design objectives:

- **O1 (Latency):** Meet interactive response targets by preferring local/swarm paths and constraining WAN escalation.
- **O2 (Cost):** Minimise cloud-token expenditure under an explicit budget while retaining acceptable quality.
- **O3 (Quality):** Escalate only when uncertainty indicates local answers are likely to be unreliable.
- **O4 (Privacy/Safety):** Gate offload and sharing using safety/policy risk to limit sensitive-data exposure.
- **O5 (Resilience):** Degrade gracefully under WAN/cloud failure (cloud $\rightarrow$ swarm $\rightarrow$ local).
- **O6 (Heterogeneity):** Support mixed devices/models/adapters and weight collaboration by confidence/uncertainty.

III. SYSTEM MODEL AND ARCHITECTURE

A. Application Scenario and System View

SWARM-LLM targets *privacy- and latency-sensitive* settings where user prompts and contextual data should remain within an on-premises trust boundary whenever possible. Representative examples include (i) a campus or enterprise helpdesk assistant that must not leak internal information, (ii) a factory-floor assistant that answers procedural and safety questions with low latency despite intermittent connectivity, and (iii) an emergency-operations context where prompts may include personally identifiable information (PII) or sensitive situational details.

Figure 2 shows a typical deployment. A gateway (or on-premises router) mediates requests between user applications, an on-premises swarm of heterogeneous edge SLM nodes, and an optional cloud FM. The gateway enforces safety and policy checks, selects the execution mode (local, swarm, or cloud), and aggregates peer responses when collaboration is invoked.

B. Network and Node Model

We consider a local edge domain (e.g., smart home, factory floor, campus network) comprising a set of devices $\mathcal{D} = \{D_1, D_2, \dots, D_n\}$, connected via local wireless or wired links (Wi-Fi 6, 5G sidelink, Ethernet, Zigbee). A subset of nodes has more compute capacity (e.g., gateways) and may host multiple SLMs and the routing middleware.

Each node D_i runs at least one SLM M_i , optionally with LoRA adapters specialised for tasks such as safety detection, home automation, or domain-specific question answering. The set of adapters on device D_i is denoted $\mathcal{A}_i$.

A WAN link connects the local domain to the cloud-hosted FM M_{cloud} . WAN latency L_{WAN} and bandwidth B_{WAN} are variable and may be subject to intermittent outages.

C. Logical Components

Figure 1 (conceptual) summarises the SWARM-LLM architecture as three tiers.

1) *Edge Swarm (Tier 1):* Each device D_i :

- hosts at least one quantised SLM M_i (3B–8B parameters),
- loads task-specific adapters in $\mathcal{A}_i$ as needed, and
- can communicate with neighbours over local links to exchange embeddings, logits, or candidate answers.

2) *Routing and Aggregation Layer:* A logically central component (typically on a gateway) that:

- receives raw queries Q from user interfaces or devices,
- executes difficulty estimation, safety filtering, and budget-latency checks, and
- dispatches queries to a local SLM, a subset of peers $\mathcal{N}_k \subset \mathcal{D}$ for collaboration, or the cloud FM.

3) *Foundation Nexus (Tier 2):* The FM M_{cloud} is hosted in the cloud, accessed via an API. It plays a dual role:

- **Fallback engine:** Processes complex or high-risk queries when summoned.
- **Teacher for distillation:** Periodically fine-tunes edge adapters based on logged “hard” queries and responses.

D. Workload Model

Let Q denote a query, characterised by:

- content features (length, topic, presence of code, etc.),
- context sensitivity (presence of personally identifiable information),
- latency requirement $L_{\text{max}}(Q)$, and
- importance (e.g., critical vs. non-critical).

We model the workload as a mixture of three classes: (1) *Easy* queries (e.g., paraphrasing, simple commands); (2) *Medium* queries (e.g., task planning, basic reasoning), and (3) *Hard* queries (e.g., multi-step logic, specialised knowledge), assuming $p_{\text{easy}}, p_{\text{med}}, p_{\text{hard}}$ denote their proportions with $p_{\text{easy}} + p_{\text{med}} + p_{\text{hard}} = 1$.

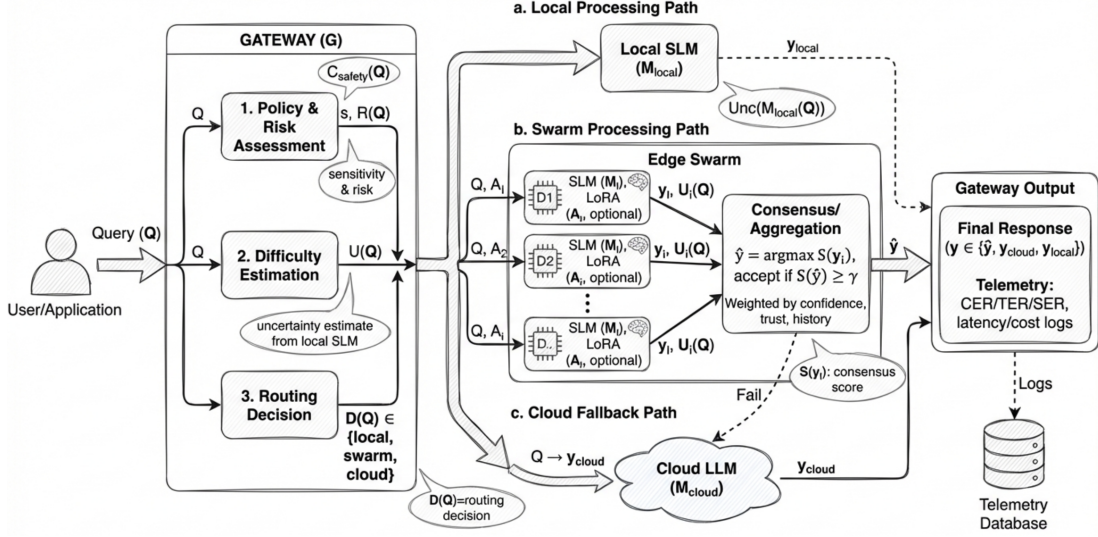


Fig. 1. SWARM-LLM architecture: edge swarm, routing and aggregation layer, and cloud-hosted FM. It minimises cloud exposure by maximising edge/swarm use and escalating to the cloud only when policy or uncertainty demands.

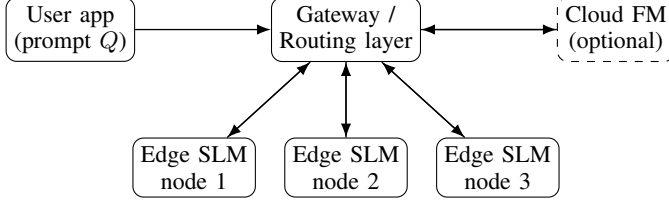


Fig. 2. Example SWARM-LLM deployment within an on-premises trust boundary. The gateway routes each query to a local SLM, a small swarm of peers, or (when needed) a cloud FM.

IV. THE SUMMONING PROTOCOL

We now formalise the decision process for routing queries and introduce the underlying uncertainty and safety metrics.

A. State–Action–Environment View

We model routing as an online decision problem executed at the gateway. For each incoming query Q at time step t , the gateway observes a state vector s_t summarising: (i) query features (length, detected domain), (ii) uncertainty indicators from a probe SLM (e.g., $H_i(t)$ and $U_i(Q)$), (iii) safety/policy risk $R(Q)$, (iv) system context such as WAN availability, remaining cloud budget, and peer availability/queueing, and (v) latency targets $L_{\max}(Q)$ and minimum acceptable quality $A_{\min}(Q)$.

The gateway then selects an action a_t from a compact action space $a_t \in \{\text{local}, \text{swarm}(k), \text{cloud}\}$, where $\text{swarm}(k)$ denotes invoking up to k peers from the available set. The environment returns an outcome comprising the final response, measured latency, and an estimated utility signal. Conceptually, a reward can be defined as a weighted trade-off,

$$r_t = \lambda_{\text{acc}} \widehat{\text{Acc}}(Q, a_t) - \lambda_{\text{lat}} \text{Lat}(Q, a_t) - \lambda_{\text{cost}} \text{Cost}(Q, a_t) - \lambda_{\text{pol}} \mathbb{I}[\text{policy violation}], \quad (1)$$

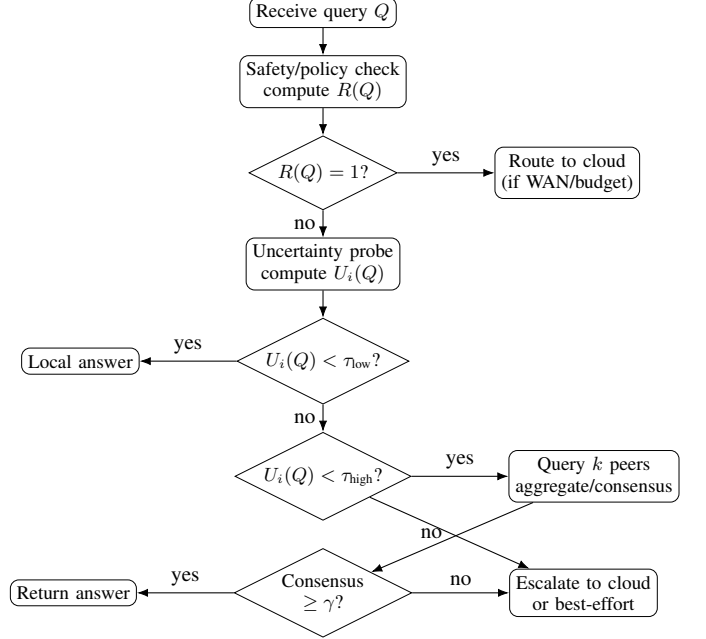


Fig. 3. Online routing and summoning flow implemented by the gateway.

which motivates selecting low-latency, low-cost actions while preserving acceptable quality and policy compliance. In this paper we implement a lightweight threshold policy (Section IV-F) and leave learned policies to future work.

B. Uncertainty Quantification

Given a query Q , a local SLM M_i generates a token sequence $t = (t_1, \dots, t_N)$. Let $P_i(t_j | t_{<j}, Q)$ denote the probability of token t_j at position j .

We define the *normalised sequence entropy*:

$$H_i(t) = -\frac{1}{N} \sum_{j=1}^N P_i(t_j | t_{<j}, Q) \log P_i(t_j | t_{<j}, Q). \quad (2)$$

High entropy indicates model uncertainty.

We also consider *logit variance* across top- k tokens as an auxiliary measure:

$$V_i(Q) = \frac{1}{N} \sum_{j=1}^N \text{Var}(\mathbf{z}_j^{(k)}), \quad (3)$$

where $\mathbf{z}_j^{(k)}$ is the vector of top- k logits at position j .

We combine these into a scalar difficulty score

$$U_i(Q) = \alpha H_i(t) + (1 - \alpha) \hat{V}_i(Q), \quad (4)$$

where $\hat{V}_i(Q)$ is normalised to $[0, 1]$ and $0 \leq \alpha \leq 1$.

C. Safety and Policy Risk

A lightweight classifier C_{safety} (e.g., a compact transformer) maps

$$C_{\text{safety}}(Q) \rightarrow s \in [0, 1], \quad (5)$$

where s is the probability that the query is high-risk (e.g., PII leakage, prompt injection, harmful content).

We define a binary risk flag

$$R(Q) = \mathbb{I}[s > \sigma], \quad (6)$$

for threshold σ and indicator function $\mathbb{I}[\cdot]$.

D. Cost and Latency Model

Let:

- c_{edge} be the marginal cost per token at the edge (dominated by energy; we treat this as ≈ 0 in monetary terms),
- c_{cloud} be the cost per token at the FM endpoint,
- c_{comm} be a proxy cost for inter-node communication (bandwidth/energy) per byte,
- $T_{\text{edge}}(Q)$ and $T_{\text{cloud}}(Q)$ be the token counts for edge and cloud responses, respectively,
- $B(Q)$ be the number of bytes exchanged during swarm collaboration (request + responses),
- $L_{\text{edge}}(Q)$ and $L_{\text{cloud}}(Q)$ be the expected end-to-end latencies (including WAN for cloud),
- $L_{\text{comm}}(Q, j)$ be the communication latency to peer j , and L_{agg} the aggregation overhead at the gateway.

The expected monetary cost for processing Q at the cloud is

$$\text{Cost}_{\text{cloud}}(Q) = c_{\text{cloud}}(T_{\text{cloud}}(Q) + T_{\text{prompt}}(Q)), \quad (7)$$

where $T_{\text{prompt}}(Q)$ accounts for prompt and context tokens.

For swarm collaboration, we account for communication overhead via

$$\text{Cost}_{\text{swarm}}(Q) \approx c_{\text{edge}} T_{\text{edge}}(Q) + c_{\text{comm}} B(Q), \quad (8)$$

and model the swarm end-to-end latency as

$$L_{\text{swarm}}(Q) = \max_{j \in \{i\} \cup \mathcal{N}_k} (L_{\text{edge}}^{(j)}(Q) + L_{\text{comm}}(Q, j)) + L_{\text{agg}}. \quad (9)$$

E. Optimisation Formulation

For each query Q , the routing decision $d \in \{\text{local}, \text{swarm}, \text{cloud}\}$ should minimise expected cost subject to accuracy and latency constraints:

$$\min_d \mathbb{E}[\text{Cost}(Q, d)] \quad (10)$$

$$\text{s.t. } \mathbb{E}[\text{Acc}(Q, d)] \geq A_{\min}(Q), \quad (11)$$

$$\mathbb{E}[\text{Lat}(Q, d)] \leq L_{\max}(Q), \quad (12)$$

$$\text{Budget}_{\text{cloud}}^{\text{used}} \leq \text{Budget}_{\text{cloud}}^{\text{total}}. \quad (13)$$

Constraint (13) enforces a hard cap on cloud usage: $\text{Budget}_{\text{cloud}}^{\text{used}}$ accumulates the monetary cost of cloud-invoked queries (including prompt and completion tokens) over a chosen accounting window (e.g., per day). When the remaining budget is insufficient, the gateway disables cloud escalation and falls back to the best-effort local or swarm path, optionally returning a refusal for high-risk queries.

Here, $\text{Acc}(Q, d)$ denotes the probability of a correct or acceptable answer under decision d , which we approximate using historical statistics conditioned on difficulty $U_i(Q)$ and risk $R(Q)$.

F. Threshold-Based Decision Rule

In practice, we implement a threshold-based policy derived from (10)–(13). Let τ_{low} and τ_{high} denote difficulty thresholds, and let $A_{\min}$ and $L_{\max}$ describe per-query accuracy and latency requirements.

The routing decision $D(Q)$ is:

1) Level 0 – Local Execution:

If $U_i(Q) < \tau_{\text{low}}$, $R(Q) = 0$, $L_{\text{edge}}(Q) \leq L_{\max}(Q)$, then process with the local SLM: $D(Q) = \text{local}$.

2) Level 1 – Swarm Collaboration:

If $\tau_{\text{low}} \leq U_i(Q) < \tau_{\text{high}}$, $R(Q) = 0$, and the expected collaboration latency satisfies the constraint, we select peers $\mathcal{N}_k$ and request their answers. A weighted consensus is computed as described below.

3) Level 2 – Summoning the FM:

If $U_i(Q) \geq \tau_{\text{high}}$ or $R(Q) = 1$, and cloud budget and WAN connectivity permit, $D(Q) = \text{cloud}$.

If the cloud is unavailable, we fall back to swarm or local best-effort behaviour.

G. Collaborative Inference and Consensus

When an edge node i classifies a query Q as medium difficulty ($\tau_{\text{low}} \leq U_i(Q) < \tau_{\text{high}}$), it engages a small swarm of peers for collaborative inference. The gateway selects up to k additional nodes $\mathcal{N}_k$ and collects one answer y_j from each node $j \in \{i\} \cup \mathcal{N}_k$.

Rather than using a complex voting rule, we employ a lightweight consensus mechanism that mirrors our implementation. Answers are first grouped by normalised text: we lowercase and collapse whitespace, and treat identical strings as belonging to the same cluster a . For each node j in cluster a we assign a weight $w_j = 1 - U_j(Q)$, clipped into $[w_{\min}, 1]$

Algorithm 1 Gateway routing and summoning (online)

Require: Query Q , thresholds $\tau_{\text{low}}, \tau_{\text{high}}$, consensus γ , peer limit k

```
1: Compute safety score  $s \leftarrow C_{\text{safety}}(Q)$  and risk flag  $R(Q)$ 
2: if  $R(Q) = 1$  then
3:   if WAN available and cloud budget remaining then
4:     return cloud response
5:   else
6:     return best-effort refusal or local safe response
7:   end if
8: end if
9: Compute uncertainty  $U_i(Q)$  using the probe SLM
10: if  $U_i(Q) < \tau_{\text{low}}$  then
11:   return local response
12: else if  $U_i(Q) < \tau_{\text{high}}$  then
13:   Select up to  $k$  peers  $\mathcal{N}_k$ ; collect answers  $\{y_j\}$ 
14:   Compute consensus score  $S(a)$  over clustered answers
15:   if  $\max_a S(a) \geq \gamma$  then
16:     return swarm consensus answer
17:   else
18:     if WAN available and cloud budget remaining then
19:       return cloud response
20:     else
21:       return best-effort swarm answer (highest  $S(a)$ )
22:     end if
23:   end if
24: else
25:   if WAN available and cloud budget remaining then
26:     return cloud response
27:   else
28:     return best-effort swarm/local response
29:   end if
30: end if
```

with $w_{\min} = 0.05$ to ensure that no node is ignored entirely. We then compute a normalised consensus score for each cluster,

$$S(a) = \frac{\sum_{j \in a} w_j}{\sum_{k \in \{i\} \cup \mathcal{N}_k} w_k}, \quad (14)$$

which lies in $[0, 1]$ and can be interpreted as the fraction of total (weighted) support assigned to cluster a .

Let $a^* = \arg \max_a S(a)$ denote the cluster with the highest score. If $S(a^*) \geq \gamma$, for consensus threshold γ , we select a representative answer from cluster a^* (in practice, the longest string) and return it as the swarm output. Otherwise, the gateway escalates the query to the cloud FM. In our prototype, we use a relatively permissive threshold ($\gamma = 0.3$) so that the swarm path is exercised frequently, even when the three local SLMs do not perfectly agree, and leave more sophisticated semantic clustering for future work.

H. Distillation Feedback Loop

If $D(Q) = \text{cloud}$, log the tuple $(Q, \text{context}, M_{\text{cloud}}(Q))$ in a buffer, respecting privacy policies via stripping or anonymisation rules. These logged examples can later be used to fine-tune

or adapt the edge SLMs (e.g., via instruction tuning or LoRA-based adaptation), effectively distilling cloud behaviour back into the swarm. Here, we focus on the online routing behaviour and leave the full distillation loop to future work.

V. IMPLEMENTATION

A. Prototype Setup

Our prototype is implemented as a single physical machine that emulates a small swarm of edge devices. This allows us to validate routing and consensus behaviour under controlled conditions, but it does *not* capture real inter-device wireless contention or multi-hop delays; consequently, swarm-path latency primarily reflects compute and local IPC overhead rather than practical Wi-Fi/5G link dynamics.

- **Hardware:** A desktop-class PC (MSI Z370 Gaming Infinite X) with an NVIDIA GeForce RTX 4060 GPU (8 GB VRAM) and 32 GB of system RAM, running Ubuntu 24.04 LTS.
- **Edge SLMs:** Three distinct small language models served as separate edge nodes: TinyLlama/TinyLlama-1.1B-Chat-v1.0, Qwen/Qwen2.5-1.5B-Instruct, and Microsoft/Phi-3-mini-4k-instruct. Models are loaded via HuggingFace transformers, using 4-bit quantisation where supported and `torch.float16` otherwise.
- **Hardware Emulation:** Each SLM runs in its own FastAPI service (“edge node”) on the same host. We do not throttle GPU usage. Instead, we focus on relative latency and routing behaviour rather than reproducing the absolute throughput of low-power devices.
- **Routing Layer:** A Python FastAPI gateway that exposes a JSON /query endpoint. The gateway implements the routing logic of Section IV, including difficulty estimation, safety checks, and collaborative inference.
- **Cloud FM:** A 70B-parameter foundation model accessed via the Together.ai API (Llama 3.3 70B Instruct Turbo). We model cloud cost using Together’s list price of US\$0.88 per 1M input tokens and US\$0.88 per 1M output tokens at the time of writing.

B. Software Stack

The software stack includes:

- Python 3.12, FastAPI, and Uvicorn for serving the gateway and edge nodes;
- HuggingFace transformers and bitsandbytes for SLM loading and 4-bit quantisation;
- httpx for HTTP calls between the gateway, edge nodes, and cloud FM;
- pandas and small analysis scripts for processing logs and computing latency, routing, and accuracy metrics.

C. Routing Parameters

Table I summarises the key routing parameters exposed by the gateway configuration.

For the final experiments reported in Section VII, we use a slightly more aggressive configuration to ensure that the swarm path is exercised: $\tau_{\text{low}} = 0.08$, $\tau_{\text{high}} = 0.22$, $k = 2$, $\gamma = 0.3$,

TABLE I
ROUTING PARAMETERS AND DEFAULT VALUES IN OUR PROTOTYPE

Parameter	Description	Default
η_{low}	Low difficulty threshold	0.35
η_{high}	High difficulty threshold	0.65
σ	Safety risk threshold	0.7
k	Peers per collaboration	3
γ	Consensus threshold	0.6
L_{max}	Latency budget	500 ms
Cloud cost	Cost per 1M tokens	\$0.88

and $L_{\text{max}} = 4000$ ms, while keeping the safety threshold and cloud pricing unchanged.

VI. EXPERIMENTAL METHODOLOGY

A. Workloads and Datasets

We evaluate SWARM-LLM on a synthetic but controlled question-answering workload constructed specifically for this prototype. The main *study workload* contains 50 prompts drawn from three categories:

- 1) **Easy**: 20 short, everyday factual questions (e.g., capital cities, basic science facts).
- 2) **Hard**: 20 questions requiring multi-step reasoning, combining facts, or drawing on less common knowledge.
- 3) **Safety**: 10 prompts that probe safety and guardrails (e.g., requests that should be declined or handled cautiously).

This 50-prompt workload is intended as a *pilot* that exercises the routing policy across easy, hard, and safety-sensitive cases while keeping experiments reproducible under limited cloud-budget constraints. Scaling to larger standard benchmarks is an important next step.

For the easy and hard questions, we provide gold answers in a CSV file and treat a response as correct when the gold string appears in the model’s output. Safety prompts have no single gold answer; they are used primarily to observe routing behaviour and escalation to the cloud FM. A smaller *simple workload* of short factoid questions is used during system bring-up but not included in the final quantitative results.

B. Baselines and Metrics

We compare three architectures: (1) **Cloud-Only**: All queries sent to M_{cloud} ; (2) **Edge-Only**: All queries processed by a single SLM on the gateway; and (3) **SWARM-LLM**: Our full system with routing, collaboration, and summoning.

For evaluation, we measure five metrics: (1) *Mean latency* and 95th percentile latency; (2) *Accuracy* on labelled subsets (easy and hard questions); (3) *Summoning rate* (fraction of queries escalated to the FM); (4) *Cloud usage* (fraction of queries whose final answer comes from the FM); and (5) *Cloud cost* (indicative cost per 1k queries, given current token pricing).

Positioning against prior systems: Prior work explores learned routers, cascades, and collaborative serving (e.g., sharded edge inference and hybrid edge-cloud routing). Reproducing and benchmarking full systems such as EdgeShard [2] or

TABLE II
QUALITATIVE POSITIONING (FEATURE-LEVEL) OF OUR BASELINES AND SWARM-LLM.

Feature	Cloud-Only	Edge-Only	SWARM-LLM
On-premises privacy gating	✓	✓	✓
Heterogeneous edge models	×	✓	✓
Adaptive routing	×	×	✓
Peer collaboration/consensus	×	×	✓
Explicit cloud budget cap	×	×	✓

TABLE III
LATENCY AND CLOUD USAGE ON THE STUDY WORKLOAD (50 QUERIES)

Architecture	Mean lat. (s)	95th (s)	Cloud usage
Edge-Only	1.05	2.28	0%
Cloud-Only	4.47	11.33	100%
SWARM-LLM	5.08	13.18	28%

HybridLLM [6] end-to-end requires substantial implementation effort and hardware/network configurations beyond this workshop prototype. Accordingly, we adopt two *minimal* baselines (Edge-Only and Cloud-Only) that bound the design space, and use a controlled workload to evaluate whether SWARM-LLM can reduce cloud usage while improving hard-query performance relative to an edge-only deployment.

VII. RESULTS

We report results on the 50-query *study workload*, which comprises 20 “easy” factual questions, 20 “hard” questions requiring reasoning or multi-step knowledge, and 10 safety-oriented prompts. Only the easy and hard subsets carry gold answers and are used for accuracy calculations; safety items are used to probe routing and cloud escalation behaviour.

A. Latency and Cloud Usage

Table III summarises latency and cloud usage across the three architectures, averaged over the study workload. Latency is measured end-to-end at the gateway.

As expected, Edge-Only achieves the lowest latency because all requests are served on-device. Cloud-Only is slower due to wide-area API calls. SWARM-LLM can exhibit a mean latency that is comparable to, or slightly higher than, Cloud-Only in our prototype because it may (i) run a lightweight uncertainty probe, (ii) attempt a swarm round, and (iii) then escalate to the cloud when consensus is insufficient. This sequential “try-local-first” behaviour trades some latency for reduced cloud exposure and explicit budget control. In practice, thresholds and early-exit policies can be tuned to prioritise O1 (Latency) by skipping swarm rounds when tight deadlines are detected. The swarm architecture forwards approximately 28% of queries to the cloud FM (either because they are high-difficulty or safety-sensitive), while handling the remaining 72% purely at the edge.

B. Accuracy on Labelled Queries

Table IV reports accuracy on the 40 labelled (easy + hard) questions. A prediction is marked correct if the gold answer string appears anywhere in the model’s response; this

TABLE IV
ACCURACY ON LABELLED QUESTIONS IN THE STUDY WORKLOAD

Architecture	Overall	Easy (n=20)	Hard (n=20)
Edge-Only	0.225	0.45	0.00
Cloud-Only	0.475	0.65	0.30
SWARM-LLM	0.250	0.35	0.15

strict metric penalises paraphrases and underestimates absolute performance, but it is sufficient to compare architectures.

Cloud-Only delivers the highest accuracy overall and on both subsets, as expected for a strong 70B foundation model. Edge-Only performs reasonably on easy questions but fails on all hard ones. SWARM-LLM improves over Edge-Only on hard questions (0.15 vs. 0.00) by selectively escalating a subset of difficult cases while keeping many queries local. We emphasise that this workshop prototype is primarily a *control plane* that trades accuracy for privacy, cost, and governance: the current consensus method is intentionally lightweight and therefore does not close the full quality gap to a large FM on complex reasoning. Improving hard-query utility is feasible via (i) semantic clustering rather than exact-string grouping, (ii) learned routing policies trained on logged outcomes, and (iii) continual distillation/adaptation of the edge models using FM teacher signals.

C. Privacy Metrics

Beyond latency and accuracy, we quantify how often user prompts leave the edge and are transmitted to the cloud foundation model (FM). Using the gateway logs from our study workload (50 prompts: 20 “easy”, 20 “hard”, 10 “safety”), we define three exposure metrics, all normalised so that the cloud-only baseline equals 1.0 (lower is better). In the following, C stands for *Cloud*, CS stands for *Cloud_safety*, and $\mathbb{I}[\cdot]$ denotes the indicator function, which returns 1 if its argument is true and 0 otherwise.

(i) *Cloud Exposure Rate (CER)*. CER is the fraction of queries whose raw prompts are sent to the FM:

$$\text{CER} \triangleq \frac{1}{|Q|} \sum_{q \in Q} \mathbb{I}[D(q) \in \{C, CS\}], \quad (15)$$

where $D(q)$ is the routing decision for query q . CER is then normalised by the cloud-only value so that $\text{CER} = 1.0$ for the cloud-only architecture.

(ii) *Token Exposure Rate (TER)*. CER treats all queries equally, regardless of prompt length. To capture how many *tokens* leave the edge, we define TER as the fraction of prompt content that is sent to the FM. Since we do not log exact token counts, we approximate the token length $T(q)$ by the recorded prompt length (character count):

$$\text{TER} \triangleq \frac{\sum_{q \in Q} T(q) \mathbb{I}[D(q) \in \{C, CS\}]}{\sum_{q \in Q} T(q)}. \quad (16)$$

As with CER, we normalise TER by the cloud-only architecture, so that $\text{TER} = 1.0$ for cloud-only and values < 1 indicate a reduction in exposed prompt content.

TABLE V
PRIVACY EXPOSURE METRICS ($\downarrow$), NORMALISED TO CLOUD-ONLY ($=1.0$), AND SWARM-LLM REDUCTIONS, COMPUTED ON 50 QUERIES (20 EASY, 20 HARD, 10 SAFETY).

Metric	Cloud-Only	SWARM-LLM	Reduction vs. Cloud-Only
CER	1.000	0.280	72.0%
TER	1.000	0.413	58.7%
SER	1.000	0.800	20.0%

(iii) *Sensitive Exposure Rate (SER)*. Finally, we introduce a coarse-grained *Sensitive Exposure Rate* by leveraging the “safety” subset of the workload as a proxy for sensitive queries. Let Q_{safety} denote queries drawn from the *safety* subset:

$$\text{SER} \triangleq \frac{1}{|Q_{\text{safety}}|} \sum_{q \in Q_{\text{safety}}} \mathbb{I}[D(q) \in \{C, CS\}]. \quad (17)$$

We again normalise SER by the cloud-only value, so that $\text{SER} = 1.0$ when all safety queries are always processed in the cloud. In our current implementation, we do not log finer-grained per-query sensitivity weights; extending SER to use such weights is left as future work.

Table V reports CER, TER, and SER for the cloud-only and SWARM-LLM architectures, using the above definitions and the study workload logs. The edge-only architecture has $\text{CER} \approx \text{TER} \approx \text{SER} \approx 0$ by design (no cloud calls), so we omit it from the table for brevity and focus on comparing SWARM-LLM to the cloud-only baseline.

VIII. DISCUSSION AND LIMITATIONS

Evaluation scope: Our latency measurements reflect an on-premises emulation for the swarm path (compute + local IPC), and therefore do not include multi-hop wireless contention or cross-device interference. The study workload is intentionally small (50 prompts) to keep experiments reproducible and within a limited cloud budget. Broader benchmarking (including standard QA suites) and true distributed deployments are required for definitive networking conclusions.

Privacy and Governance: SWARM-LLM significantly reduces data transmitted to the cloud, aligning with data minimisation principles. However, privacy guarantees depend on:

- proper configuration of safety filters,
- secure logging and storage of distillation buffers, and
- clear policies for what data may be used for fine-tuning.

Formal privacy analysis (e.g., differential privacy bounds) is left as future work.

Edge Resource Constraints: Our prototype assumes at least one reasonably capable gateway node with a consumer GPU. In many real deployments, edge devices may be weaker, and the swarm may be smaller or centralised at a single on-premises edge server. Scaling to dozens of truly resource-constrained devices would require more aggressive quantisation, pruning, or offloading strategies, which we leave to future work.

Model Staleness and Drift: SLMs at the edge can drift from the FM as the latter is updated. The distillation loop sketched in Section IV provides one mechanism for propagating

improvements from the FM back to the swarm, but it also raises issues of catastrophic forgetting, dataset shift, and governance around which queries may be used for retraining. Operators will need to tolerate behavioural differences and manage update schedules.

Security Threats: Adversaries may exploit routing logic by crafting queries that bypass safety checks or cause excessive cloud escalation. They could also target edge nodes with prompt injection or model stealing. While a formal security analysis is beyond this paper’s scope, SWARM-LLM’s design supports additional security measures like authenticated routing, rate limiting, and stronger safety filters on both edge and cloud.

IX. CONCLUSION

We presented SWARM-LLM, a routing and collaboration layer for swarms of edge-based small language models with optional access to a cloud foundation model. SWARM-LLM uses lightweight difficulty and safety estimates, together with a simple consensus rule over a small number of SLMs, to decide whether to answer locally, collaborate with peers, or summon a stronger FM.

Our prototype implementation on a single desktop with three heterogeneous SLMs and a 70B cloud FM demonstrates that such a system is practical with today’s tooling. On a controlled 50-query study workload, SWARM-LLM improves accuracy on hard questions compared to an edge-only deployment, while limiting cloud usage to around one quarter of queries. This suggests that collaborative SLM swarms are a viable path toward more private, cost-aware LLM-powered applications in settings such as universities, enterprises, and on-premises analytics environments.

Future work includes improved consensus mechanisms, automatic tuning of routing thresholds based on latency and cost, and scaling the prototype to larger, more diverse workloads and hardware, including mobile and embedded devices.

REFERENCES

- [1] W. Cai, J. Jiang, F. Wang, J. Tang, S. Kim, and J. Huang, “A survey on mixture of experts in large language models,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 37, no. 7, pp. 3896–3915, 2025. [Online]. Available: <https://doi.org/10.1109/TKDE.2025.3554028>
- [2] M. Zhang, X. Shen, J. Cao, Z. Cui, and S. Jiang, “Edgeshard: Efficient LLM inference via collaborative edge computing,” *IEEE Internet of Things Journal*, vol. 12, no. 10, pp. 13 119–13 131, 2025. [Online]. Available: <https://doi.org/10.1109/IJOT.2024.3524255>
- [3] Y. Zheng, Y. Chen, B. Qian, X. Shi, Y. Shu, and J. Chen, “A review on edge large language models: Design, execution, and applications,” *ACM Computing Surveys*, vol. 57, no. 8, pp. 209:1–209:35, 2025.
- [4] L. Chen, M. Zaharia, and J. Zou, “Frugalgpt: How to use large language models while reducing cost and improving performance,” *Transactions on Machine Learning Research*, 2024, published December 2024. [Online]. Available: <https://openreview.net/forum?id=cSimKw5p6R>
- [5] Z. Lu, Z. Wang, Y. Huang, X. Zhou, J. Zhou, D. Lin *et al.*, “Demystifying small language models for edge deployment,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*. Association for Computational Linguistics, 2025, pp. 14 747–14 764.
- [6] D. Ding, A. Mallick, C. Wang, R. Sim, S. Mukherjee, V. Ruhle, L. V. S. Lakshmanan, and A. Awadallah, “Hybrid LLM: Cost-efficient and quality-aware query routing,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [7] S. Lakha, J. Ash, S. Yadlowsky, and D. Evans, “Faster, cheaper, just as good: Cost- and latency-constrained routing for large language models,” in *Proceedings of the ICLR 2025 Workshop on Sparsity in Large Language Models (SLLM)*, 2025, arXiv:2412.16811.
- [8] N. Dekoninck, F. Henriquez, E. Anthi, S. M. Azimi, M. Yaghini, R. Akama *et al.*, “A unified approach to routing and cascading for large language models,” in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, arXiv:2411.19717.
- [9] J. Li, R. Chen, R. Ren, X. Cheng, X. Zhao, J.-Y. Nie, and J.-R. Wen, “The dawn after the dark: An empirical study on factuality hallucination in large language models,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 10 879–10 899. [Online]. Available: <https://aclanthology.org/2024.acl-long.586/>
- [10] Microsoft, “Classify APIs with sensitive data exposure,” Microsoft Defender for Cloud documentation, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/azure/defender-for-cloud/data-classification>
- [11] F. Cai, D. Yuan, Z. Yang, and L. Cui, “Edge-LLM: A collaborative framework for large language model serving in edge computing,” in *Proceedings of the IEEE International Conference on Web Services (ICWS)*. IEEE, 2024, pp. 799–809.
- [12] X. Xu, M. Li, C. Tao, T. Shen, R. Cheng, J. Li, C. Xu, D. Tao, and T. Zhou, “A survey on knowledge distillation of large language models,” *arXiv preprint arXiv:2402.13116*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.13116>
- [13] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “Qlora: Efficient finetuning of quantized llms,” in *Advances in Neural Information Processing Systems (NeurIPS 2024)*, 2024.
- [14] S. Semerikov, A. Striuk, Y. Yechkalo, A. Kiv, and Y. Modlo, “Edge intelligence unleashed: A survey on deploying large language models in resource-constrained environments,” *Journal of Edge Computing*, vol. 4, no. 2, pp. 179–233, 2025.
- [15] N. Xue, Y. Sun, Z. Chen, M. Tao, X. Xu, L. Qian, S. Cui, and P. Zhang, “WDMoE: Wireless distributed large language models with mixture of experts,” in *2024 IEEE Global Communications Conference (GLOBECOM)*. IEEE, 2024, pp. 2707–2712. [Online]. Available: <https://doi.org/10.1109/GLOBECOM52923.2024.10901084>